

PDS - CheatSheet

Python for Data Science | BE05000231 | GTU | SEM 5

Compiled BY : Manthan A. D. [in]

CH. 1. PYTHON BASICS, OPERATORS, DECISIONS, LOOPS AND FUNCTIONS

1. Important definitions

Term	Meaning
Python	High-level, general-purpose programming language.
Variable	A name referring to a value or object.
Data type	Defines a kind of value, such as `int`, `float` or `str`.
Boolean	Logical value: `True` or `False`.
Function	Reusable block of code defined using `def`.
Module	Python file containing reusable code.
Scope	Region where a variable can be accessed.

2. Essential data types

Type	Example	Key property
Integer	`10`	Whole numbers
Float	`3.14`	Decimal numbers
String	`"Python"`	Text; immutable
List	`[1, 2, 3]`	Ordered, mutable

Type	Example	Key property
Tuple	`(1, 2, 3)`	Ordered, immutable
Set	`{1, 2, 3}`	Unique elements
Dictionary	`{"name": "A"}`	Key-value pairs
Boolean	`True`	Logical value

List vs Tuple vs Dictionary. HIGH PRIORITY

- **List:** Changeable sequence. `a[0]` accesses its first element.
- **Tuple:** Sequence that cannot be modified after creation.
- **Dictionary:** Stores values against keys. Example: `student["name"]`.
- **Set:** Removes duplicates. Use `set()` to create an empty set.

3. Operators

Operator type	Examples	Purpose
Arithmetic	`+ - * / // % **`	Calculations
Assignment	`= += -= *=`	Assign or update values
Comparison	`== != > < >= <=`	Compare values
Logical	`and`, `or`, `not`	Combine conditions
Membership	`in`, `not in`	Check membership
Identity	`is`, `is not`	Check object identity
Bitwise	AND `&`; OR `|`; XOR `^`; NOT `~`; left shift `<<`; right shift `>>`	Integer bit operations

Important distinction:

```
a == b    # Values are equal
a is b    # Both refer to the same object
```

4. Decisions, loops and functions

- `if / elif / else`: Selects a branch based on conditions.
- `for`: Iterates over a sequence or iterable.
- `while`: Repeats while a condition is true.
- `break`: Exits the loop.
- `continue`: Skips to the next iteration.
- `pass`: Placeholder that does nothing.
- `range(start, stop, step)`: Generates numbers, excluding `stop`.
- **Recursion:** A function calls itself. It must have a base case to stop.

Essential snippets

```python
# String slicing
text = "Python"
print(text[1:4])       # yth

# Loop and condition
for n in range(1, 6):
    if n % 2 == 0:
        print(n)

# Function and recursion
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))    # 120

# Fibonacci series
a, b = 0, 1
for _ in range(6):
```

```
    print(a, end=" ")
    a, b = b, a + b
```

Remember: `return` sends a result back to the caller. `print()` only displays output.

Other core points: use `import math`, `import random`, and `import statistics` to access common modules. Python indentation defines code blocks.

CH. 2. FILE HANDLING, EXCEPTION HANDLING AND PYTHON-MYSQL

1. Important definitions

Term	Meaning
File handling	Reading data from or writing data to files.
File I/O	File Input/Output operations.
Exception	Error or unusual event that interrupts normal execution.
Exception handling	Handling errors so a program can respond appropriately.
CSV	Comma-Separated Values, a common tabular data format.
Binary file	Stores bytes rather than ordinary text.
Database	Organized collection of data.
SQL	Structured Query Language, used with relational databases.
Cursor	Object used to execute database operations and retrieve results.

2. File modes

Mode	Meaning
`r`	Read
`w`	Write, replacing existing content
`a`	Append to the end
`x`	Create a new file; fail if it exists
`b`	Binary modifier, e.g. `rb`

Mode	Meaning
`t`	Text modifier, e.g. `rt`

Text and CSV files

```
# Read a text file
with open("data.txt", "r", encoding="utf-8") as f:
    content = f.read()

# Write to a text file
with open("output.txt", "w", encoding="utf-8") as f:
    f.write("Hello Python")
```

`with open(...)` manages the file's lifetime and closes it automatically.

```
# Read a CSV file with Pandas
import pandas as pd

df = pd.read_csv("data.csv")
print(df.head())
```

3. Exception handling. BLIND SPOT

```
try:
    number = int(input("Enter a number: "))
    print(10 / number)
except ValueError:
    print("Enter a valid integer.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
finally:
    print("Execution completed.")
```

- `try`: Contains code that may raise an exception.

- `except`: Handles a matching exception.
- `finally`: Runs whether an exception occurs or not.
- `raise`: Manually raises an exception.

Binary file example

```
with open("source.bin", "rb") as f:
    data = f.read()

with open("copy.bin", "wb") as f:
    f.write(data)
```

4. Basic MySQL operations. BLIND SPOT

Workflow

```
Connect → Create cursor → Execute SQL → Fetch results → Close connection
```

```
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="your_user",
    password="your_password",
    database="your_database"
)

cur = conn.cursor()
cur.execute("SELECT * FROM students")
rows = cur.fetchall()

for row in rows:
    print(row)
```

```
cur.close()
conn.close()
```

Install the connector separately if needed: `pip install mysql-connector-python`.

Essential rule: For queries with user-supplied values, use parameterized SQL rather than constructing queries through string concatenation.

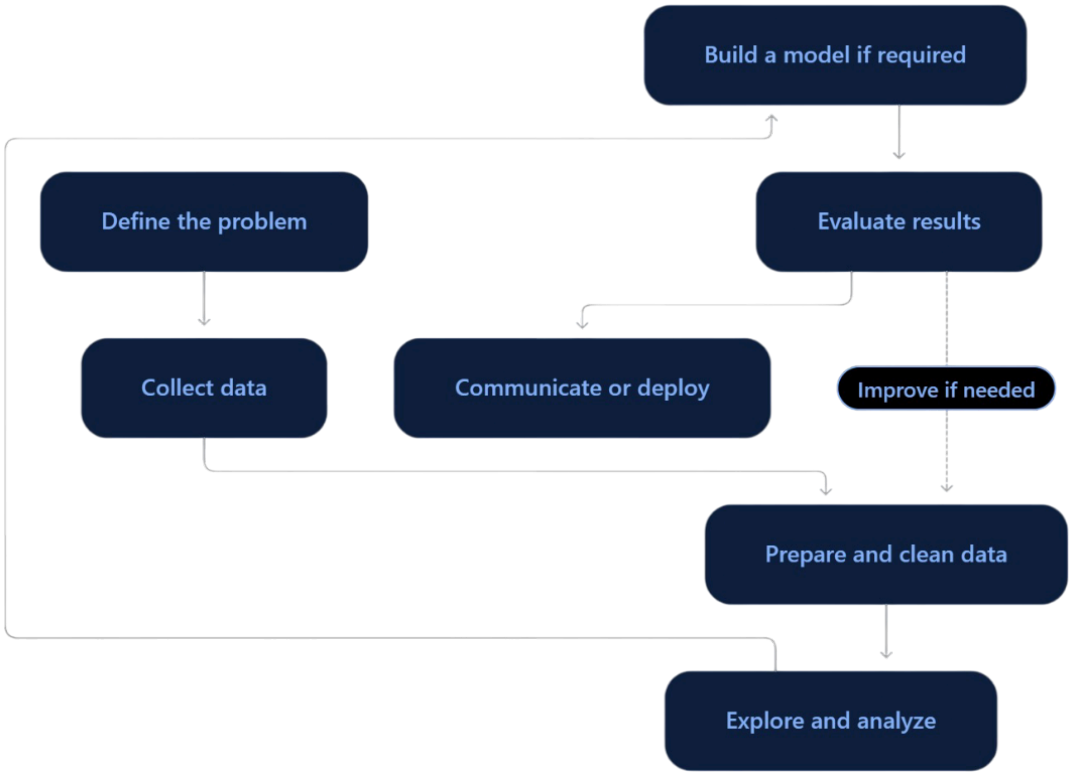
CH. 3. DATA SCIENCE AND DESCRIPTIVE STATISTICS

1. Important definitions

Term	Meaning
Data Science	Using data, statistics and computing to obtain insights and support decisions.
Data analysis	Examining data to find patterns and answer questions.
Data analytics	Applying analytical methods to support decisions.
EDA	Exploratory Data Analysis. Examining data before deeper analysis or modelling.
Descriptive statistics	Summarizes the main characteristics of observed data.
Covariance	Indicates whether two variables tend to vary together.
Correlation	Measures the strength and direction of a relationship between variables.

HIGH PRIORITY: Data Science life cycle, EDA, covariance, correlation, skewness and kurtosis.

2. Data Science life cycle



The stages may repeat. Not every data science task requires a machine-learning model.

3. Types of data analysis

- **Univariate:** Studies one variable. Example: distribution of student marks.
- **Bivariate:** Studies two variables. Example: study hours vs marks.
- **Multivariate:** Studies three or more variables together.

4. Data measurement scales. CORE

Scale	Meaning	Example
Nominal	Categories without inherent order	Blood group
Ordinal	Categories with an order	Low, medium, high
Interval	Equal intervals, no true zero	Temperature in °C
Ratio	Equal intervals and a meaningful zero	Height, weight

5. Measures of central tendency and dispersion

Measure	Meaning / Formula
Mean	Average: `sum(x) / n`
Median	Middle value after sorting
Mode	Most frequent value
Range	Maximum − minimum
Variance	Average squared deviation from mean; denominator depends on population vs sample
Standard deviation	Square root of variance
IQR	`Q3 - Q1`, spread of the middle 50%
Percentile	Value below which a given percentage of observations falls

6. Covariance and correlation

- **Covariance:** Positive means variables tend to rise together; negative means one tends to rise as the other falls.
- **Correlation coefficient** `r`**:** Ranges from `-1` to `+1`.
  - `r ≈ +1`: Strong positive linear relationship.
  - `r ≈ -1`: Strong negative linear relationship.
  - `r ≈ 0`: Little or no linear relationship.
- Correlation does **not** establish causation.

7. Skewness and kurtosis

- **Skewness:** Measures asymmetry of a distribution.
  - Positive: longer right tail.
  - Negative: longer left tail.
  - Near zero: approximately symmetric.

- **Kurtosis:** Describes tail heaviness relative to a normal distribution under the chosen definition. In Pandas, `.kurt()` returns **excess kurtosis**, for which a normal distribution has value zero.

```
print(df["marks"].mean())
print(df["marks"].median())
print(df["marks"].std())
print(df["marks"].skew())
print(df["marks"].kurt())
print(df[["hours", "marks"]].corr())
```

CH. 4. EXPLORING PYTHON LIBRARIES

1. Important definitions

Library	Main purpose
NumPy	Numerical computing and multidimensional arrays.
Pandas	Tabular data handling and analysis.
SciPy	Scientific computing, including statistical functions.
Scikit-learn	Machine-learning algorithms and utilities.
BeautifulSoup	Extracting data from HTML or XML documents.

HIGH PRIORITY: NumPy vs Pandas and Scikit-learn, including classification and regression.

2. NumPy vs Pandas

NumPy	Pandas
Focuses on arrays and numerical operations	Focuses on labelled, tabular data
Main structure: `ndarray`	Main structures: `Series`, `DataFrame`
Useful for efficient numerical calculations	Useful for filtering, cleaning, joining and summarizing tables

NumPy basics

```
import numpy as np

a = np.array([10, 20, 30])
print(a.shape)     # (3,)
print(a.mean())    # 20.0
print(a[0])        # 10
```

- `ndarray`: N-dimensional array.
- `shape`: Dimensions of an array.
- `axis=0`: Operations down rows, producing column-wise results for a 2D array.
- `axis=1`: Operations across columns, producing row-wise results for a 2D array.

Pandas basics

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.shape)
print(df.tail())
print(df.describe())
```

These common operations are expanded in CH. 6.

3. Machine learning basics

- **Machine Learning (ML):** Methods that learn patterns from data.
- **Supervised learning:** Uses labelled examples.
- **Unsupervised learning:** Finds patterns without supplied target labels.
- **Classification:** Predicts a category, such as spam or not spam.
- **Regression:** Predicts a numerical value, such as price.

Scikit-learn: minimal regression workflow

```
from sklearn.linear_model import LinearRegression

X = [[1], [2], [3], [4]]  # Feature values
y = [2, 4, 6, 8]          # Numeric targets

model = LinearRegression()
model.fit(X, y)
print(model.predict([[5]]))
```

`fit()` learns from the supplied data. `predict()` produces predictions.

4. SciPy statistics and BeautifulSoup

```
from scipy import stats

print(stats.binom.pmf(k=2, n=5, p=0.5))
```

This calculates the probability of exactly 2 successes in 5 independent trials, each with success probability 0.5.

```
from bs4 import BeautifulSoup

html = "<h1>Python</h1>"
soup = BeautifulSoup(html, "html.parser")
print(soup.h1.get_text())   # Python
```

BeautifulSoup parses HTML. Follow the target website's access rules when collecting web data.

CH. 5. PROBABILISTIC AND INFERENTIAL STATISTICS

BLIND SPOT in the supplied PYQ analysis. Study the definitions, distributions and testing procedure even though the analyzed papers had no recorded coverage.

1. Important definitions

Term	Meaning
Probability	Numerical measure of how likely an event is, from 0 to 1.
Random variable	Variable whose numerical value depends on a random outcome.
Distribution	Describes possible values of a variable and their probabilities.
Sampling	Selecting observations from a population.
Population	Entire group being studied.
Sample	Subset of that population.
Inference	Drawing conclusions about a population using sample data.
Estimation	Using sample data to estimate a population parameter.
Hypothesis test	Procedure for evaluating a claim using sample evidence.

2. Basic probability rules

Probability bounds

$$0 \leq P(A) \leq 1$$

Complement

$$P(A^c) = 1 - P(A)$$

For mutually exclusive events A and B:

$$P(A \cup B) = P(A) + P(B)$$

For independent events:

$$P(A \cap B) = P(A)P(B)$$

Remember: Mutually exclusive events cannot occur together. Independent events do not change each other's probabilities.

3. PMF vs PDF vs CDF

Function	Full form	Used for
PMF	Probability Mass Function	Discrete random variables
PDF	Probability Density Function	Continuous random variables
CDF	Cumulative Distribution Function	Both discrete and continuous variables

- **PMF:** Gives `P(X = x)`, probability at a discrete value.
- **PDF:** Describes probability density. For a continuous variable, probability is obtained by integrating over an interval.
- **CDF:** `F(x) = P(X ≤ x)`, probability of being at or below `x`.

For continuous variables, the probability of an exact single value is zero: `P(X = x) = 0`.

4. Probability distributions

Distribution	Type	Typical application
Binomial	Discrete	Number of successes in a fixed number of independent trials
Poisson	Discrete	Count of events in a fixed interval under suitable assumptions
Normal	Continuous	Bell-shaped distributions
Exponential	Continuous	Waiting time between events in a Poisson process

Essential formulas

Binomial distribution

$$P(X = k) = C(n, k)p^k(1 - p)^(n - k)$$

- `n`: Number of trials.

- `k`: Required number of successes.
- `p`: Probability of success per trial.

Poisson distribution

$$P(X = k) = (e^(- \lambda)\lambda^k)/k!$$

- `λ`: Expected event count in the interval.
- `k = 0, 1, 2, ...`

Standard normal distribution

$$Z = (X - \mu)/\sigma$$

- `μ`: Population mean.
- `σ`: Population standard deviation.
- A standard normal variable has mean 0 and standard deviation 1.

5. Sampling techniques

- **Simple random:** Every eligible individual has an equal chance of selection.
- **Systematic:** Select every `k`-th observation after a starting point.
- **Stratified:** Divide the population into strata and sample from each.
- **Cluster:** Divide the population into clusters and select clusters for sampling.

6. Estimation and the Central Limit Theorem

- **Point estimation:** Gives one estimated value. Example: sample mean = 72.
- **Interval estimation:** Gives a range of plausible values with a stated confidence level.
- **Central Limit Theorem (CLT):** Under suitable conditions, the sampling distribution of the sample mean becomes approximately normal as sample size increases, even when the population itself is not normal.

A common confidence interval for a mean when population standard deviation is known:

$$\bar{x} \pm z_(\alpha/2)\sigma/\sqrt{n}$$

This is not a universal formula for every interval. Other conditions may require a t-based interval or a different method.

7. Hypothesis testing. Essential procedure

1. State the **null hypothesis** `H0` and alternative hypothesis `H1`.
2. Choose significance level `α`, often 0.05.
3. Select an appropriate test and calculate its test statistic and p-value.
4. Compare p-value with `α`.
5. Report the conclusion in context.

Decision rule

- If `p ≤ α`: Reject `H0`.
- If `p > α`: Fail to reject `H0`.

Do not say that `H0` is definitely true just because you failed to reject it.

One-sample test using SciPy

```
from scipy import stats

sample = [52, 49, 55, 51, 53]

t_stat, p_value = stats.ttest_1samp(
    sample, popmean=50
)

print(t_stat, p_value)
```

This tests whether the sample provides evidence that its population mean differs from 50. A one-sample *t*-test is typically used when population standard deviation is unknown and assumptions are appropriate.

CH. 6. DATA PREPARATION

Highest study priority: Missing-data handling, Pandas operations, filtering, categorical variables, data wrangling and Z-score standardization.

1. Important definitions

Term	Meaning
Data preparation	Making raw data suitable for analysis.
Data cleaning	Finding and correcting or removing errors and inconsistencies.
Missing data	Observations where a value is unavailable.
Imputation	Filling missing values with estimated or selected values.
Categorical variable	Variable representing categories or groups.
Data wrangling	Restructuring and transforming data into usable form.
Outlier	Observation unusually far from the general pattern.
Normalization	Rescaling values, often to a defined range.
Standardization	Rescaling data using its mean and standard deviation.
DataFrame	Two-dimensional labelled table in Pandas.

2. Load and inspect data. HIGH PRIORITY

```
import pandas as pd

df = pd.read_csv("data.csv")

print(df.shape)          # (rows, columns)
print(df.head())         # First 5 rows
print(df.tail())         # Last 5 rows
print(df.columns)        # Column names
```

```
print(df.dtypes)          # Data types
print(df.describe())      # Numeric descriptive statistics
```

- `shape`: Tuple (number_of_rows, number_of_columns).
- `head(n)` / `tail(n)`: First or last `n` rows.
- `describe()`: Summary statistics for selected data.
- `info()`: Data types and non-null counts.

3. Selecting, slicing and filtering

```
# Select a column
df["marks"]

# Select rows/columns by labels
df.loc[0:4, ["name", "marks"]]

# Select by integer positions
df.iloc[0:5, 0:2]

# Filter using a condition
passed = df[df["marks"] >= 40]

# Multiple conditions
result = df[
    (df["marks"] >= 40) & (df["attendance"] >= 75)
]
```

- `loc`: Label-based selection. Label-based slices may include the end label.
- `iloc`: Integer-position selection. Python-style slices exclude the end position.
- Use `&` for element-wise AND between Pandas conditions and wrap each condition in parentheses.

Slicing vs dicing: Slicing selects a portion of data. Dicing selects a subset using specific values or conditions across dimensions.

4. Missing-data handling. HIGH PRIORITY

Identify missing values

```
df.isna()
df.isna().sum()
```

Common strategies

Strategy	When useful
`dropna()`	Remove rows or columns with missing values when appropriate
Mean imputation	Numeric data without severe skew or outlier problems
Median imputation	Numeric data where skew or outliers make median preferable
Mode imputation	Categorical data
Forward/backward fill	Ordered data where carrying nearby values is justified

```
# Remove rows containing missing values
df_clean = df.dropna()

# Impute a numeric column with its median
df["marks"] = df["marks"].fillna(df["marks"].median())

# Impute a categorical column with its mode
df["city"] = df["city"].fillna(df["city"].mode()[0])
```

Important: Do not automatically delete or fill every missing value. Choose the method based on the data and purpose of analysis.

5. Categorical variables and encoding

- **Nominal:** Categories without an order. Example: city.
- **Ordinal:** Categories with an order. Example: low, medium, high.
- **Label/ordinal encoding:** Converts categories to numbers. Use ordered values when the order is meaningful.

- **One-hot encoding:** Creates indicator columns for categories, usually suitable for nominal features.

```
encoded = pd.get_dummies(
    df, columns=["city"], dtype=int
)
```

6. Duplicates, sorting, grouping and combining

```
# Count and remove duplicates
print(df.duplicated().sum())
df = df.drop_duplicates()

# Sort by a column
df = df.sort_values("marks", ascending=False)

# Group and aggregate
summary = df.groupby("city")["marks"].mean()

# Combine rows from two similar tables
combined = pd.concat([df1, df2], ignore_index=True)

# Create a new variable
df["total"] = df["test1"] + df["test2"]

# Remove a column
df = df.drop(columns=["old_column"])
```

- `groupby()`: Split data into groups, apply a calculation, then combine results.
- **Aggregating:** Summarizing multiple values, such as with `sum()`, `mean()`, or `count()`.
- **Shuffling:** Randomly changes row order. It does not sort by value.

7. Outliers, normalization and standardization

IQR method for outliers

$$IQR = Q3 - Q1$$

- Lower fence: `Q1 - 1.5 × IQR`
- Upper fence: `Q3 + 1.5 × IQR`
- Values outside these fences are commonly flagged for inspection, not automatically deleted.

Min-max normalization

$$x' = (x - x_min)/(x_max - x_min)$$

Usually maps values to [0, 1], provided the maximum and minimum differ.

Z-score standardization. HIGH PRIORITY

$$z = (x - \mu)/\sigma$$

- `x`: Original value.
- `μ`: Mean used for scaling.
- `σ`: Standard deviation used for scaling.
- A Z-score shows how many standard deviations a value is from the mean.

For sample-based data, use a consistent choice of mean and standard deviation. In ML workflows, learn scaling parameters on training data only, then apply them to validation/test data to avoid leakage.

CH. 7. DATA VISUALIZATION

HIGH PRIORITY: Plot types, Matplotlib labels, annotations, legends, and scatter plots with trendlines. Seaborn basics are included as a blind spot in the PYQ analysis.

1. Important definitions

Term	Meaning
Data visualization	Presenting data graphically to identify patterns and communicate findings.
Matplotlib	Python library for creating graphs and plots.
Seaborn	Statistical visualization library built on Matplotlib.
Figure	Overall canvas that holds plots.
Axes	Region containing an individual plot, axes and labels.
Subplot	One plot among multiple plots within a figure.
Legend	Explains the meaning of plotted data series.
Annotation	Adds explanatory text to a point or region.

2. Which plot should you use?

Plot	Main purpose
Line plot	Show trends, especially over time
Bar chart	Compare categories
Histogram	Show distribution of numerical data
Pie chart	Show proportions of a whole
Box plot	Summarize spread and flag potential outliers
Scatter plot	Examine relationships between two numerical variables

Remember: A histogram groups numerical values into bins. A bar chart compares distinct categories. They are not interchangeable.

3. Essential Matplotlib snippets

Basic plot with labels and legend

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 25, 30]
```

```
plt.plot(x, y, label="Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Monthly Sales")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Bar, histogram, pie and box plots

```
# Bar chart
plt.bar(["A", "B", "C"], [20, 35, 25])
plt.title("Category Comparison")
plt.show()

# Histogram
plt.hist([12, 15, 15, 18, 20, 22], bins=4)
plt.title("Value Distribution")
plt.show()

# Pie chart
plt.pie(
    [40, 35, 25],
    labels=["A", "B", "C"],
    autopct="%1.1f%%"
)
plt.show()

# Box plot
plt.boxplot([12, 15, 15, 18, 20, 22])
plt.title("Distribution and Potential Outliers")
plt.show()
```

Important: `show()` displays the current figure. Start a new figure using `plt.figure()` when you need to create another plot in a context where figures are being reused.

4. Scatter plot with a trendline

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 5, 4, 7])

m, b = np.polyfit(x, y, 1)

plt.scatter(x, y, label="Observed")
plt.plot(x, m * x + b, label="Trendline")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Scatter Plot with Trendline")
plt.legend()
plt.tight_layout()
plt.show()
```

- `scatter()`: Displays individual observations.
- `np.polyfit(x, y, 1)`: Fits a straight line using least squares.
- `annotate()`: Adds explanatory text to a chosen point.

Example:

```
plt.annotate("Peak", xy=(3, 5))
```

5. Subplots and Seaborn

Multiple plots in one figure

```
fig, axes = plt.subplots(1, 2, figsize=(8, 3))

axes[0].plot([1, 2, 3], [2, 4, 6])
axes[0].set_title("Line Plot")

axes[1].bar(["A", "B"], [5, 8])
axes[1].set_title("Bar Chart")

fig.tight_layout()
plt.show()
```

`plt.subplots(1, 2)` creates one row and two plot areas.

Seaborn basics

```
import seaborn as sns

sns.histplot(data=df, x="marks", kde=True)
plt.show()
```

- `kde=True`: Adds a smooth estimate of the distribution's density.
- Seaborn provides convenient statistical plots. Matplotlib remains useful for detailed figure customization.

FINAL REVISION. WHAT TO STUDY FIRST

The PYQ analysis is based on previously examined papers under an updated syllabus mapping. Treat its frequencies as **revision priorities, not guaranteed predictions**.

Highest-priority revision

- Missing-data handling, imputation and encoding
- Data Science life cycle and EDA

- Pandas DataFrame operations, slicing and filtering
- NumPy vs Pandas
- Covariance, correlation, skewness and kurtosis
- Matplotlib plot types, labels and legends
- Classification vs regression
- List vs tuple vs dictionary
- Data wrangling, duplicates and Z-score standardization
- String slicing, `range()`, factorial and Fibonacci

Do not skip these syllabus areas :

> Python exceptions and file handling, MySQL basics, probability distributions, PMF/PDF/CDF, sampling, CLT, estimation, hypothesis testing, SciPy statistics, BeautifulSoup and Seaborn.

Your PYQ analysis identifies these as gaps in past-paper coverage rather than evidence that they are unimportant.

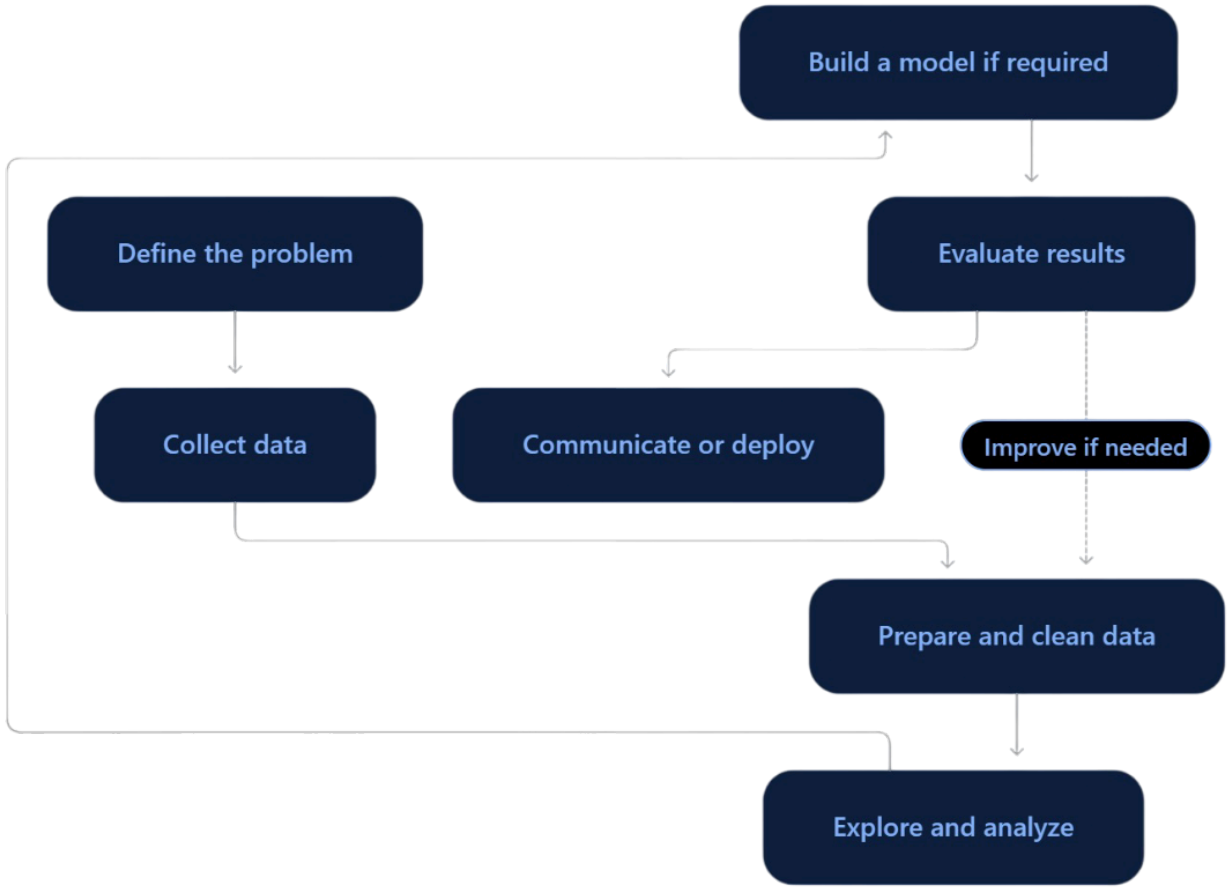
Essential keywords

```
Python · Data Types · Functions · File I/O · Exceptions
Data Science Life Cycle · EDA · Descriptive Statistics
NumPy · Pandas · SciPy · Scikit-learn
PMF · PDF · CDF · CLT · Hypothesis Testing
Imputation · Data Wrangling · Z-score
Matplotlib · Seaborn
```

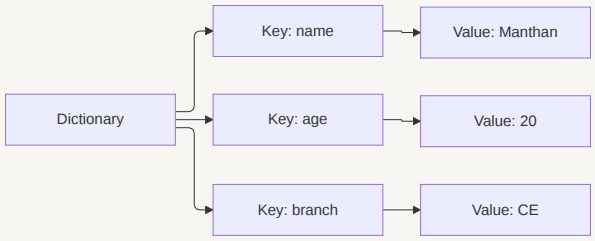
EXAM-DRAWABLE DIAGRAMS

These are simplified, exam-friendly diagrams. In Notion, each Mermaid block can be viewed as a rendered diagram; practise redrawing the same flow with boxes and arrows in your exam.

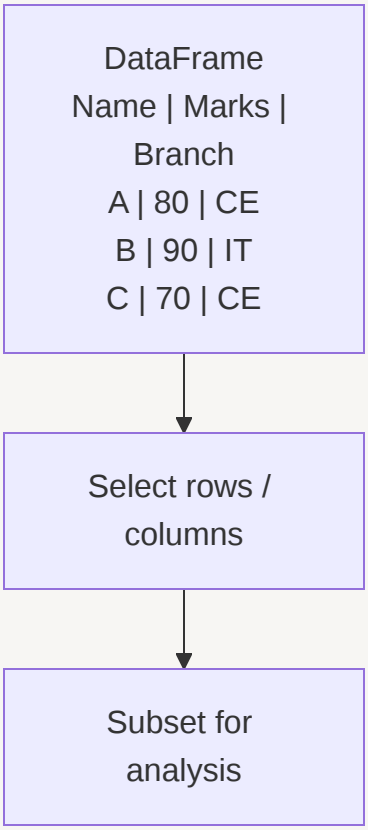
1. Data Science life cycle



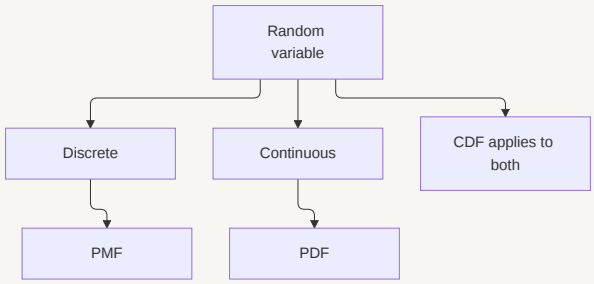
2. Dictionary key-value model



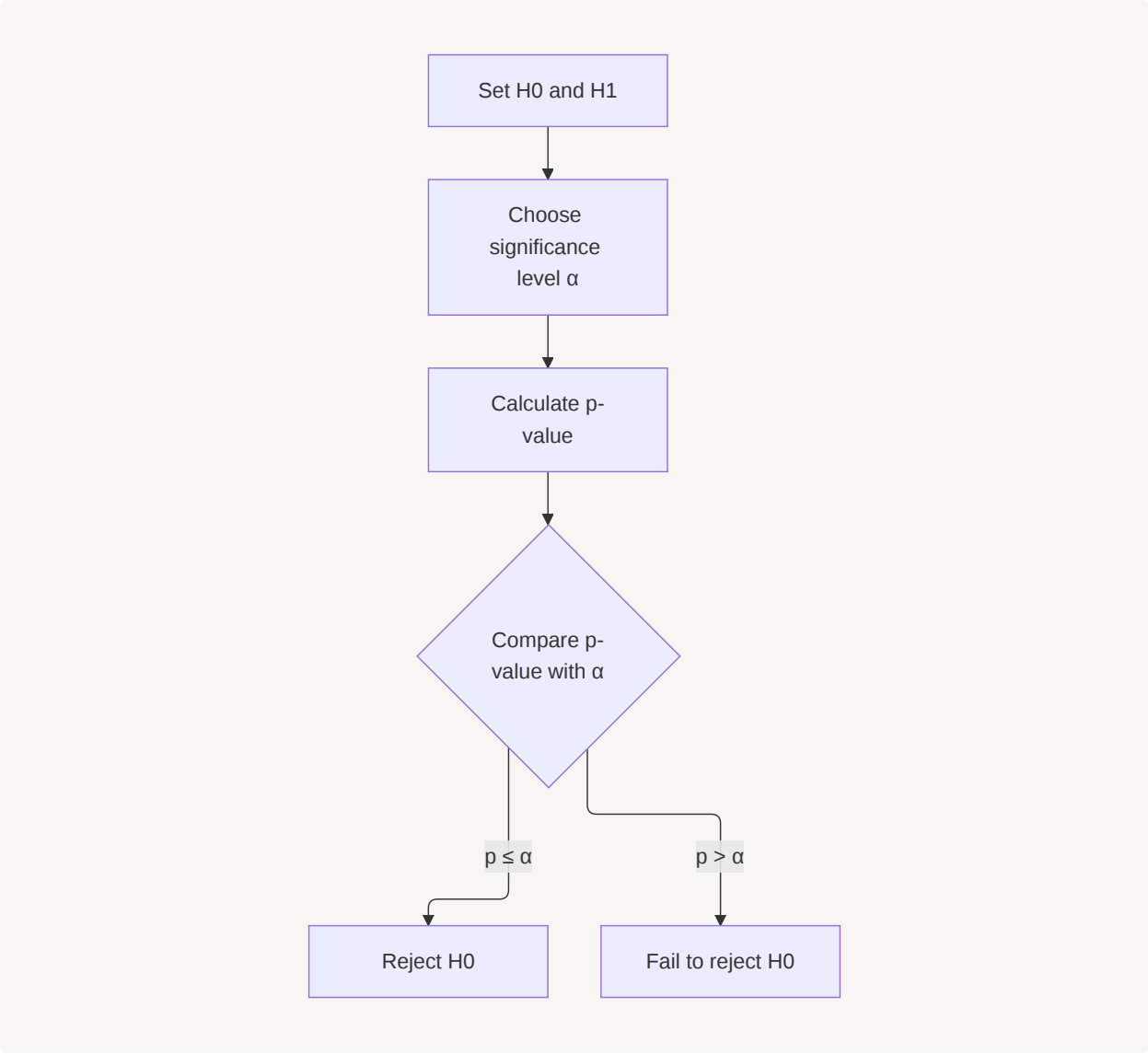
3. DataFrame selection



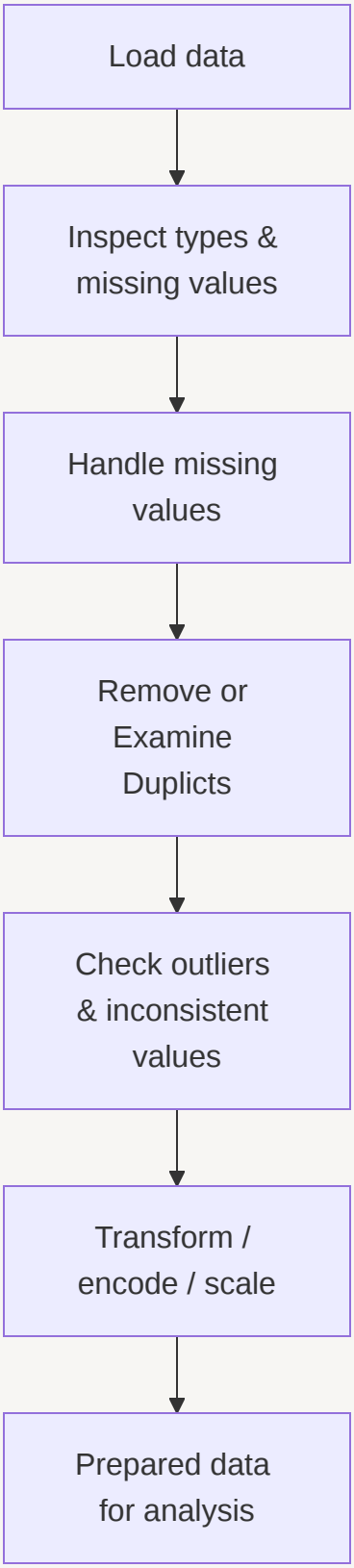
4. Probability distribution guide



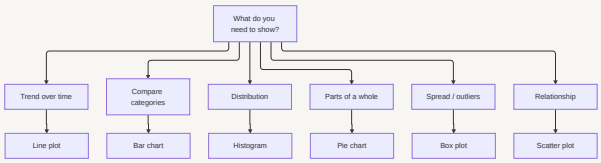
5. Hypothesis testing decision



6. Data cleaning workflow


Load data
Inspect types & missing values
Handle missing values
Remove or Examine Duplicts
Check outliers & inconsistent values
Transform / encode / scale
Prepared data for analysis

7. Visualization choice



Quick technical checkpoints

- Python provides core language features; libraries extend it for data science.
- NumPy is mainly for numerical arrays; Pandas is mainly for labelled/tabular data.
- EDA helps understand a dataset before deeper analysis or modelling.
- Classification predicts a category; regression predicts a numerical value.
- PMF is for discrete variables; PDF describes continuous density; CDF works for both.
- Correlation does not prove causation.
- `dropna()` removes missing observations; `fillna()` replaces missing values.
- `loc` is label-based; `iloc` is integer-position-based.
- Z-score standardization uses the mean and standard deviation.
- Bar charts compare categories; histograms show numerical distributions.
- A figure can contain multiple axes/subplots.
- Missing data, statistical assumptions and data leakage must be handled carefully.

Compiled by: Manthan A. D.